Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.0000000

Does Rubric-Guided Refinement Improve Automatically Optimized Prompts? A Pre-Specified Negative-Result Study with Small Open-Weight Models

LINH NGUYEN[1], QUANG-VINH DANG[2], MINH NGOC DINH[3], AND THUY NGUYEN[1]
[1]School of Science, Engineering & Technology, RMIT University Vietnam, 702 Nguyen Van Linh Boulevard, Ho Chi Minh City 700000, Vietnam
[2]British University Vietnam, Hung Yen, Vietnam
[3]Millenia Education, Ho Chi Minh City, Vietnam

Corresponding author: Quang-Vinh Dang (e-mail: vinh.dq4@buv.edu.vn)

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

ABSTRACT Large language models remain critically dependent on prompt quality, and automatic prompt optimization (APO) methods often produce output lacking the structured guidance human-crafted prompts provide. This paper reports a pre-registered go/no-go pilot testing Meta-Prompted Instruction Refinement (MPIR), a lightweight rubric-guided layer that refines an APO-generated prompt through meta-prompted evaluation, refinement, and empirical validation. An initial evaluation using GPT-4o to judge and refine prompts targeting GPT-3.5-turbo reported per-task gains on Big-Bench Hard (BBH) but a pooled improvement that did not reach significance, and a later audit found its validation stage had leaked training data into scoring. To isolate whether the apparent gains reflected the refinement mechanism or the meta-model's capability advantage over its target, we reran the pilot with a single small open-weight model (Qwen3-1.7B) as target, optimizer, and refinement judge simultaneously, across three APO backbones on the three BBH tasks where the original gains were largest. The effect did not reproduce: MPIR's refined prompt was byte-identical to its base optimizer's prompt in all nine tested configurations, and pooled McNemar and GEE tests returned accuracy differences of 0.0000. Tracing the cause, we identify and fix a general defect in the refinement algorithm – it never scored the pre-refinement prompt as a baseline – plus six further correctness bugs found live. We report this as a boundary-condition finding: a cheap, post-hoc rubric-refinement layer of this kind appears to depend on the refining model's own capability, and its benefit is smaller than the original per-task deltas suggest.

INDEX TERMS Big-Bench Hard, Prompt engineering, automatic prompt optimization, large language models, meta-prompting, negative results

I. INTRODUCTION

The prompt is the specification, the interface and the test fixture of an LLM-based component at once, and unlike source code it cannot be checked by a compiler. Large language models (LLMs) perform strongly on tasks such as translation [1], summarization [2] and question answering [3] (see [4] for a survey), yet small changes in phrasing or formatting can change their accuracy substantially [5], [6]. Prompt design has therefore become a bottleneck for reliable deployment in domains such as law and medicine [7], [8].

Two families of approaches address that bottleneck. Manual prompting applies heuristics distilled from practice, such as chain-of-thought reasoning [9], role assignment [10] and few-shot demonstration [11]; these are interpretable but need expertise and trial and error [12], [13]. Automatic prompt optimization (APO) searches the space of prompts with the help of an LLM, using critique, mutation, evolution or optimizer-style proposal loops [14]–[18]; it scales but may forfeit the structure that human-written prompts carry [19]. A natural idea, which we call Meta-Prompted Instruction Refinement

(MPIR), is to add a cheap post-hoc stage after any APO run: ask an LLM to score the optimized prompt against a rubric of manual heuristics, ask it to rewrite the prompt accordingly, and keep the rewrite only if it scores better on held-out validation questions.

This paper does not propose MPIR as a new method. It reports a pre-specified test of whether that idea works when the same small open-weight model plays every role, and what the evidence does and does not allow us to conclude. The study has a history that readers need in order to weigh it. An earlier evaluation used GPT-4o as the refining model and GPT-3.5-turbo as the target on 23 Big-Bench Hard (BBH) tasks [20]. It reported per-task gains over PromptWizard [18] on a majority of tasks but a pooled gain that was not statistically significant (Appendix D reproduces those numbers). A later code audit found two defects: the validation stage scored candidate prompts on the optimizer's own training examples, and the refinement loop never scored the unrefined prompt, so a refinement was always accepted whatever it scored. A manuscript describing that evaluation was desk-rejected by another journal without external review. The numbers in it should be treated as exploratory and are not used as evidence here.

We therefore asked a narrower question that the first evaluation could not answer: are the apparent gains attributable to the rubric-guided refinement mechanism, or to the refining model being more capable than the model it refines for? A first pilot with Qwen3-1.7B in every role found the refined prompt byte-identical to the base prompt in all nine (task, optimizer) cells. Review of that pilot, however, exposed weaknesses that we take seriously: the seven refinement rounds were not independent attempts, the composition of the outcomes was not reported, the statistics were degenerate, the task choice was biased, the capability-gap explanation was untested, and the evaluation harness floored several baselines. This revision answers each of them with new experiments, specified in a version-controlled protocol before they were run (Section 4.1).

We ask four research questions. RQ1: with the refinement loop repaired so that rounds are genuinely distinct attempts, does gated MPIR improve a base optimizer's prompt over many tasks, optimizers and seeds? RQ2: is the failure to improve a property of the candidates or of the coarse validation gate? RQ3: does a more capable refining model change the answer? RQ4: does the rubric judge measure anything related to accuracy? @@SUMMARY@@

The contributions are:

- A repaired and instrumented refinement loop. Rounds differ by sampling and, in the chained variant, by the history of rejected candidates; every round is logged with an outcome category (fallback, echo-rejected, duplicate, validated), its validation and test scores, and the judge's rubric total.
- A pre-specified, multi-seed, multi-task evaluation. 45 base prompts (three optimizers, nine tasks, up to three seeds), with decision rules fixed in a time-stamped protocol and a random, documented choice of six tasks to complement the three pilot tasks.
- Diagnostics that make a null result interpretable. Oracle scoring of all candidates on the test split, acceptance-rule sensitivity, a serving-noise floor and minimum detectable effect, a gated free-rewrite control, an uncorrected-algorithm comparison, and a judge-validity analysis against a stronger reference judge.
- Harness findings. Strict answer extraction floors some baselines; lenient re-scoring of the saved raw outputs and a catalogue of the pipeline defects, with serving configuration, are reported (Appendix C).

Section 2 reviews related work. Section 3 describes the layer under test. Section 4 gives the design and statistics. Section 5 reports results, Section 6 discusses what they support and their limits, and Section 7 concludes. Code, per-example predictions and per-round logs are public.

II. RELATED WORK

A. MANUAL PROMPTING HEURISTICS

Prompting steers an LLM through in-context learning without changing its weights [11], [21], and has been described as a form of natural-language programming [13]. Non-expert authors struggle because they get no structural feedback and over-fit to the few examples they try [12]. Experienced authors rely on heuristics. Few-shot demonstrations help [11], [22], though Min et al. show that format, label space and input distribution matter more than label correctness [23]. Chain-of-thought prompting decomposes multistep problems [9], [24], and coherent, relevant reasoning steps matter more than length [25], [26]. Role prompting assigns a persona to guide reasoning [10], but its benefit is inconsistent: poorly aligned personas can hurt [27], and a large study found that adding personas to system prompts does not reliably improve performance [28]. Step-back prompting abstracts key principles before solving [29]; a behavioural benchmark of 35 LLMs found that it fosters model-based behaviour in sequential decision tasks [30], which is weaker support for knowledge-intensive reasoning than the original step-back evidence. Models attend unevenly to information in long contexts [31], [32], and vendor guidance reports recency effects [33]; whether either matters for the short prompts studied here is an empirical question rather than a premise. Self-consistency, a decoding-time heuristic, samples several reasoning paths and votes [34]; MPIR does not use it.

B. AUTOMATIC PROMPT OPTIMIZATION AND META-PROMPTING

APO methods generate, evaluate and select prompts iteratively [19]. APE induces instructions from input-

output demonstrations [14]; ProTeGi treats LLM critique as a textual gradient and runs a beam search [15]; EvoPrompt applies evolutionary operators [16]; OPRO conditions an LLM on a trajectory of scored prompts [17]; PromptWizard combines critique-based refinement, example selection and self-generated reasoning [18]. DSPy, TextGrad, Self-Refine and Reflexion generalize the idea of optimizing or refining text with LLM feedback [20], [35]–[37]. Meta-prompting supplies an LLM with a template that structures the critique (see [38] for a survey): PE2 uses a step-by-step diagnosis template [39], dual-phase methods add a meta-instruction-guided initialization [40], and PROPEL injects expert-derived principles as priors [41]. Closest in spirit to MPIR, a two-stage approach for few-shot relation extraction first searches and then locally edits the winner [42]. More recent work explores evolutionary reflection [43], error taxonomies [44], format optimization [45], debate-based fitness [46] and RL over edits [47], and several studies target small or open-weight models: gradient-guided optimization with small models [48], minimal-edit optimization for on-device models [49], and error-driven optimization of arithmetic reasoning [50]. Others distill reasoning behaviour into prompts [51], regularize prompts for token efficiency [52], question what drives optimizer gains [53], or codify principled instructions [54].

C. LIMITS OF SELF-REFINEMENT AND OF LLM JUDGES

MPIR asks a model to critique and rewrite its own prompt, which places it in the self-refinement family [36], [37]. Evidence on intrinsic self-correction is sobering: LLMs cannot reliably self-correct reasoning without external feedback [55], and small models need strong verifiers to benefit from correction [56]. MPIR's critic is also an LLM judge, and LLM judges show position, verbosity and self-preference biases [57]–[59]. Finally, benchmark accuracy depends on prompt format and on how answers are extracted [6], [60], [61], which matters here because MPIR's gate and our metrics both rely on a delimiter-based extraction. These strands predict that a 1.7B model refining its own prompts may fail for reasons unrelated to the rubric, which is why our design varies the refiner, validates the judge and re-scores answers leniently.

D. REPORTING NEGATIVE RESULTS

Reproducibility programmes [62] and guidance on reporting experimental results [63] call for budgets, variance and full protocols. Variance across seeds and data splits can exceed the effect of a method [64], significance testing in NLP needs care [65], and small test sets leave studies under-powered [66]. We follow this guidance by fixing decision rules in advance, reporting per-cell paired differences with intervals rather than tests that cannot be informative, and quantifying the noise floor and the minimum detectable effect.

E. POSITIONING

Table 1 contrasts how the methods above propose and select candidates. It states design differences, not evidence of superiority: whether any of them outperforms the others is not tested here.

TABLE 1: Design comparison of MPIR with representative methods. The table describes design differences only; it reports no empirical comparison.

Method	Candidate generation	Selection	Role relative to an APO run
APE ‹19›	Induction from demonstrations	Execution accuracy	Optimizer
ProTeGi ‹20›	Textual-gradient edits	Beam search on minibatches	Optimizer
EvoPrompt ‹21›	Crossover and mutation	Fitness per generation	Optimizer
OPRO ‹22›	LLM proposal given score history	Best of trajectory	Optimizer
PromptWizard ‹23›	Critique and sequential re-optimization	Scores on training batches	Optimizer
PE2 ‹24›	Structured diagnosis template	Held-out scoring	Optimizer
PROPEL ‹26›	Expert principles as priors	Implicit in the priors	Optimizer
GEPA ‹59›	Evolution with reflection	Reflection-guided selection	Optimizer
MPIR (tested here)	Rubric-guided critique and rewrite	Validation accuracy vs. the unrefined prompt	Post-hoc layer

What distinguishes MPIR in this table is its role, not a mechanism: it adds one more critique-rewrite-validate cycle on top of whatever an upstream method produced. That makes it cheap and optimizer-agnostic, but also means its mechanism is close to existing critique-and-revise loops [15], [36], [39]; we do not claim novelty beyond the combination of an explicit rubric with a post-hoc, gated role.

III. THE MPIR LAYER UNDER TEST

MPIR is a three-stage cycle applied to a prompt P0 that an upstream APO method has already produced: (1) a language model scores P0 against a seven-criteria rubric of manual prompting heuristics and recommends improvements; (2) it rewrites P0 accordingly; (3) the rewrite is scored on validation questions and kept only if it beats the incumbent (Figure 1). We describe it here as the object under test, not as a contribution.

A. PROBLEM FORMULATION

Let (Qv, Av) = {(qi, ai)}, i = 1..M, be a validation set of M question-answer pairs and A(L, P) the accuracy of the target model L on it under prompt P. Let R(P0) be the set of refinements that the refining model L' produces from P0. MPIR returns

Pfinal = argmax over P' in {P0} ∪ R(P0) of A(L, P'),

with ties resolved in favour of P0 (a candidate must be strictly better). Including P0 in the candidate set is what makes it possible for the procedure to conclude that no refinement helps.

B. ALGORITHM AND VARIANTS

Algorithm 1 is the repaired loop. It scores P0 first, so the incumbent score Abest starts at A(L, P0) instead of minus infinity. In the original implementation the incumbent score started at minus infinity, so the first round's candidate was accepted whatever it scored, which is the defect Section 5.8 quantifies. Choosing among {P0} ∪ candidates by held-out score is standard in APO loops, so we regard this as an implementation defect of ours, not as a general property of the method.

Algorithm 1 MPIR with a validation gate (repaired)

Require: L: target LLM; L': refining LLM; Meval, Mrefine: meta-prompts; R: rubric; P0: prompt from an APO run; (Qv, Av): validation set; N: rounds; mode in {independent, chained}; τ: sampling temperature
Ensure: Pfinal

```
Pbest ← P0; Abest ← Validate(L, P0, (Qv, Av)); H ← empty list   ▷ score the baseline first
for t = 1 to N do
    Psrc ← P0 if mode = independent else Pbest
    S ← Evaluate(Meval, L', Psrc, R; τ)   ▷ rubric critique
    P' ← Refine(Mrefine, L', Psrc, S, H; τ)   ▷ H is empty when independent
    outcome ← Classify(P')   ▷ fallback, echo, duplicate or validated
    if outcome = validated then
        A' ← Validate(L, P', (Qv, Av)); append (P', A') to H
        if A' > Abest then Pbest ← P'; Abest ← A'   ▷ strict: a tie keeps the incumbent
    end if
end for
return Pbest
```

The pilot ran what we call the legacy loop: Psrc = P0 in every round, H empty and $\tau = 0$. Because Evaluate and Refine then receive identical input in every round, the seven rounds were one request repeated, up to serving non-determinism, and 'seven rounds without acceptance' meant roughly one distinct attempt per cell. The repaired loop differs between rounds in two ways: independent mode samples at $\tau = 0.7$ so each round is a fresh draw, and chained mode refines the current incumbent and shows the model the earlier candidates with their validation scores, as OPRO does with its trajectory [17]. We study five variants (Table 2).

TABLE 2: Refinement variants studied. All use the same gate (strict improvement over the incumbent on 25 validation questions) and N = 7 rounds.

Variant	Refines	History	Sampling	Rubric
V0 legacy (as in the pilot)	P0	none	greedy	original
V1 independent	P0	none	$\tau = 0.7$	correcte
V2 chained (primary)	incumbent	scored candidates	$\tau = 0.7$	correcte
V2L chained	incumbent	scored candidates	$\tau = 0.7$	original
V3 gated free rewrite	P0	none	$\tau = 0.7$	none (n rubric)

C. THE SEVEN-CRITERIA RUBRIC

The rubric was assembled from the prompting literature [67], [68], industry guidance [33], [69]–[71] and iterative experimentation on BBH-style tasks. It scores a prompt from 1 to 5 on: (1) Role Prompting, (2) Step-Back Reasoning, (3) Guided Chain of Thought, (4) Instruction and Separation, (5) Output Format with Examples, (6) Worked Reasoning in Examples and (7) Conclusion, for a total out of 35. The evidence behind the criteria is uneven. Chain-of-thought and demonstrations are well supported [9], [23], [26]. Role prompting is the least supported, since its benefit is inconsistent [27], [28]. The Conclusion criterion extrapolates positional-bias results from long contexts [31], [32] to short prompts, and the vendor page we cite for it states that instruction placement made no difference for GPT-4-class models [33]. Industry sources are practitioner guidance rather than empirical evidence; they are living documents, so the reference list gives archived copies and access dates.

The rubric text used in the pilot was internally inconsistent: it announced 'five defined criteria', asked for 'all 6 criteria' and 'Total Score: X/25' while listing seven criteria. We found this during the audit. The primary variants use a corrected text (seven criteria, X/35); variant V2L keeps the original text as a sensitivity check.

D. EVALUATION, REFINEMENT AND OUTPUT GUARDS

The evaluation meta-prompt (Figure 2) asks the refining model to apply the rubric to the prompt and report scores, strengths, weaknesses and 7-10 suggestions. The refinement request (Figure 3) asks for a rewrite wrapped in <START> and <END>; in chained mode it also lists the earlier candidates with their validation accuracy and asks the model not to repeat them. Three output guards exist, and each outcome is logged per round. A response without delimiters is a fallback: it cannot be validated,

and the pilot's code returned P0 in that case, which ties the baseline by construction. A candidate containing phrases verbatim from the evaluation template, or APE's induction template, is an echo and is rejected before validation. A candidate textually identical to the incumbent, to P0 or to an earlier candidate is a duplicate. Only the remaining candidates are validated.

```
### Task Overview
You are a Senior Prompt Engineer with
two main tasks:
1. Evaluate prompts using a
7-criteria rubric. 2. Refine prompts
by applying evaluation feedback.
### Prompt Under Evaluation
[Prompt]: {prompt}
### Instructions
1. Review Prompt 2. Apply Rubric
(seven defined criteria)
3. Score & Justify: score 1-5, one
strength, one weakness, a rationale
per criterion
4. Calculate Total: cumulative score
out of 35
5. Recommend 7-10 actionable
improvements 6. Report in the
standardized format
### Seven-Criteria Evaluation Rubric
1. Role Prompting 2. Step Back
3. Guided Chain of Thought 4.
Instruction & Separation
5. Output Format with Examples 6.
Worked Reasoning in Examples 7.
Conclusion
### Output Format
1. Role Prompting - X/5 - Strength:
... - Improvement: ... - Rationale:
...
(continue for all 7 criteria) **Total
Score: X/35** Refinement Summary: ...
```

FIGURE 1: Evaluation meta-prompt (corrected text; abridged from the repository file).

```
Refine the prompt by applying all
suggestions from the evaluation.
Make sure to wrap the refined prompt
with <START> and <END>

[chained mode adds: Earlier
refinement attempts ... did NOT beat
the current best prompt on the
validation questions; do not repeat
them: Attempt k (validation accuracy
a vs current best b): ...]
```

FIGURE 2: Refinement request used by the independent (top) and chained (bottom) variants.

E. THE VALIDATION GATE

A candidate is accepted only if its accuracy on the 25 validation questions strictly exceeds the incumbent's. With 25 questions accuracy moves in steps of 4 percentage points, and one question of serving noise equals a step, so acceptance is rare by design. Whether the gate or the candidates explain the absence of accepted refinements is research question RQ2, which we answer by scoring every candidate on the full test split (the oracle analysis) and by re-applying other acceptance rules offline (Section 5.4).

IV. EXPERIMENTAL DESIGN

A. PRE-SPECIFICATION AND DECISION RULES

The first pilot's go/no-go rule was written in the project's planning document (REBUILD.md, commit 7612aa0, 2026-09-03) before the pilot ran. It was a version-controlled plan, not an entry in a public registry, and the pilot was not pre-registered in the strict sense in which we used that word in the first submission. Several pipeline fixes were also made while the pilot was running (Appendix C). For this revision we wrote the protocol in REVISION_PROTOCOL.md and committed it (commit 8fd1397, 2026-10-19) before running any revised experiment; its commit time-stamps the rules below, which are reproduced from it. Dated amendments are listed in Appendix F. This is also not a registry entry.

- D1 (RQ1). Primary outcome per cell: paired difference in test accuracy between the final MPIR prompt and P0 (10,000-resample paired bootstrap, 95% percentile interval); a cell is a win if the difference is positive and the interval excludes zero. The effect 'reproduces' if at least 15 of 45 cells are wins and the mean cell difference is at least +2.0 points; it is 'null' if at most 2 of 45 are wins and the mean lies within ±1.0 point; otherwise 'mixed'.
- D2 (RQ3). Capability dependence is supported if the mean cell difference with the 8B refiner exceeds that with the 1.7B refiner by at least 2.0 points and at least 3 of 9 cells are wins with the 8B refiner.
- D3 (RQ2). The gate is limiting if the best oracle candidate beats P0 on test by at least 3.0 points in at least 3 of 9 cells although the gated final prompt did not.

McNemar, GEE and Wilcoxon tests on identical prompts cannot be informative, and the Wilcoxon test on three tasks cannot go below $p = 0.25$, so none is reported as inference. Rubric-accuracy correlations, lenient parsing, the uncorrected-algorithm comparison, the noise floor and the minimum detectable effect are descriptive.

B. DATA, SPLITS AND TASK SELECTION

BBH has 23 tasks in 27 task files, about 6,511 examples [20]. Each task is split three ways with random.Random(seed).sample over the original example order: 25 optimizer-training examples, 25 validation examples for the gate, and the remainder as test (200

examples for every task used here except penguins_in_a_table, which has 96). The partition is stored with the original example indices, is shared by all optimizers and conditions for a given (task, seed), and no stage touches test data during optimization or refinement. Tier 1 is the three pilot tasks (hyperbaton, ruin_names, penguins_in_a_table) at seeds 42, 43 and 44. They were chosen because the leaked first evaluation showed the largest gains there, so they are a biased sample and may be subject to winner's-curse effects. Tier 2 is six tasks drawn uniformly at random, without replacement, from the other 24 task files with random.Random(20261019): temporal_sequences, movie_recommendation, disambiguation_qa, geometric_shapes, salient_translation_error_detection and sports_understanding, at seed 42. In total there are 45 cells, one per (task, optimizer, seed), each with its own P0.

C. MODELS AND SERVING

The target model is Qwen3-1.7B [72] with thinking disabled and greedy decoding, at most 1024 new tokens. The refining model is the same model in the main grid and Qwen3-4B-AWQ or Qwen3-8B-AWQ in the capability experiment; meta-prompt calls allow 1500 (evaluation) and 3000 (refinement) new tokens. Refiner sampling uses temperature 0.7. Everything is served with vLLM 0.28.0 under WSL2 on one 16 GB RTX 5000 Ada GPU, bf16 weights for the 1.7B model, a maximum context of 8192 tokens (Appendix C). Greedy decoding is not bit-reproducible under continuous batching, so we measure the resulting noise (Section 5.7).

D. BASE OPTIMIZERS AND CONDITIONS

APE and ProTeGi are clean-room implementations of the published algorithms [14], [15], run with few_shot_count = 0, hence exemplar-free; ProTeGi scores every successor on a full minibatch instead of using the original bandit scheme. PromptWizard [18] is run with synthetic-example generation disabled; its three in-context examples are drawn from the 25 training examples with model-generated reasoning, and its generated expert identity is not used at evaluation. Hyperparameters are in Table 3. Only the expert few-shot condition uses the three BBH-author exemplars [20]; the first manuscript's statement that APE and ProTeGi also used them was wrong. Reference conditions are zero-shot CoT [24] and expert few-shot CoT. The free-rewrite control (Appendix A.3) rewrites P0 with no rubric; V3 adds the gate.

E. METRICS

Accuracy is the share of test examples whose delimited answer equals the gold answer after removing parentheses, whitespace and case (strict extraction, as in the pilot). Because strict extraction penalizes correct reasoning that is wrapped badly, we add a lenient accuracy computed offline from the saved raw outputs: the response counts as correct if the delimited span (or, if there is none, the last 300 characters) names exactly one option and it is the gold one. Both are reported; strict extraction remains primary because MPIR's gate uses it.

TABLE 3: Hyperparameters of the base optimizers and of MPIR as actually used.

Component	Parameter	Description	Value used
PromptWizard	mutate_refine_iterations	Outer rounds of mutate-then-refine over the task description	3
PromptWizard	mutation_rounds	Style-mutation rounds inside one outer round	3
PromptWizard	style_variation	Prompt variants generated per call	5
PromptWizard	refine_task_eg_iterations	Critique rounds on instruction and examples	3
PromptWizard	few_shot_count / seen_set_size	In-context examples / training examples	3 / 25
APE	num_candidates, iterations	Induced candidates; resampling rounds	10, 2
APE	num_scoring_examples, top_n	Scoring minibatch; candidates kept	5, 3
ProTeGi	beam_width, num_steps	Beam width; expansion rounds	3, 3
ProTeGi	minibatch_size, eval_batch_size	Error minibatch; scoring batch	5, 8
MPIR	N, validation size	Rounds; gate questions	7, 25
MPIR	refiner temperature, max tokens	Sampling; evaluation / refinement	0.7, 1500 3000
MPIR	history kept (chained)	Last candidates shown; characters each	4, 800
MPIR	guards	Fallback, echo (5 phrases), duplicate	see Section 3.4

F. STATISTICAL ANALYSIS

For every cell we report the paired difference in test accuracy between the final MPIR prompt and P0 with a 95% paired-bootstrap interval over test examples. Cells with identical prompts have a difference of zero by construction, apart from serving noise. Across cells we report counts of wins, the mean difference and a hierarchical bootstrap interval that resamples tasks, then optimizers and seeds within tasks. The noise floor is the distribution of accuracy differences between repeated scorings of the same prompt, and the minimum detectable effect at 80% power and $\alpha = 0.05$ follows from the paired discordance rate [65], [66].

G. HARNESS DEFECTS AND DEVIATIONS

Running against a live model exposed defects that mocked tests could not (truncation of long responses, answer normalization, delimiter mismatches in expert exemplars, template mismatches between an optimizer's own and MPIR's evaluation harness, two echo patterns and the unscored baseline). All are described with their magnitude in Appendix C and were fixed before the numbers reported here were produced.

V. RESULTS

PLACEHOLDER

VI. DISCUSSION

PLACEHOLDER

VII. CONCLUSION

PLACEHOLDER

APPENDIX. PROMPT EXAMPLES

A. BASE PROMPTS PRODUCED BY A 1.7B-DRIVEN OPTIMIZER

These are prompts exactly as saved by the pipeline (seed 42), not cleaned. They show the quality of the upstream, 1.7B-driven baselines that MPIR was asked to refine, which limits how informative refining them can be. PromptWizard's prompt for penguins_in_a_table opens with a detached question that is not a task instruction; the APE prompt for hyperbaton is APE's own instruction-induction meta-prompt, not a task instruction, because the search selected a candidate that echoed the induction template; the APE and ProTeGi prompts for the other two tasks are the seed instruction "Let's think step by step." plus the answer-format line.

```
How could I determine the accuracy of
my solution to this problem?

[Question] Here is a table where the
first line is a header and
each subsequent line is a penguin: name,
age, height (cm),
weight (kg) Louis, 7, 50, 11 Bernard,
5, 80, 13 Vincent, 9, 60, 11 ...
What is the name of the 60 cm tall penguin?
Options: (A) Louis (B) Bernard (C) Vincent
(D) Gwen (E) James
[Answer] ... Therefore, the name of the
60 cm tall penguin is Vincent.
**Final Answer:** (C) Vincent <ANS_START>(C)<ANS
END>
[... two further worked examples omitted
...]
Wrap the final answer between <ANS_START>
and <ANS_END>
```

Figure A.1. PromptWizard's saved prompt for penguins_in_a_table (abridged).

```
I gave a friend an instruction. Based
on the instruction they
produced the following input-output pairs:
[Question] Which sentence has the correct
adjective order: ...
[Answer] (B)
[... further input-output pairs omitted
...]
What was the instruction I gave?
Wrap the final answer between <ANS_START>
and <ANS_END>
```

Figure A.2. APE's saved prompt for hyperbaton (abridged): an unextracted induction meta-prompt, not a task instruction.

B. REFERENCE PROMPTS

The zero-shot condition uses the BBH authors' task description plus "Let's think step by step" and the answer-format line. The expert few-shot condition appends the authors' three worked chain-of-thought exemplars [20], each followed by a bare <ANS_START>(X)<ANS_END> tag. This tag is the property that exemplar-free prompts lack (Section 5.1).

```
Q: Which of the following is a humorous
edit of this artist or movie name: 'whitesnake'?
Options: (A) whitesnape (B) whitesnapke
(C) whitesnuake (D) mwhitesnake
A: Let's think step by step. ... So the
answer is (A).
<ANS_START>(A)<ANS_END>
```

Figure A.3. First expert exemplar for ruin_names, abridged. The same exemplar is the single format exemplar used in the intervention of Section 5.1.

C. FREE-REWRITE CONTROL PROMPT

```
You are given a task-solving prompt generated
for a Big-Bench Hard task.
Rewrite the prompt to improve clarity,
readability, and instruction organization
while
preserving the original task meaning.
Preserve the original task, answer format,
and examples.
Do not use any prompt-evaluation rubric.
Output only the rewritten task-solving
prompt.
Original prompt: <START> {prompt} <END>
```

Figure A.4. Meta-prompt of the free-rewrite control (abridged).

APPENDIX. ECHO FAILURE TRANSCRIPTS

Two ways in which a small refiner failed silently, found by inspecting saved outputs after an anomalous accuracy reading in the pilot. Both are well-formed <START>/<END> responses with no usable task instruction. They are now caught by the echo guard (Section 3.4) and, independently, by the validation gate.

```
<START>
I gave a friend an instruction. Based
on the instruction they produced the
following
```

```
input-output pairs: [Question] Which
sentence has the correct adjective order:
...
The task is to determine the correct
adjective order in a sentence ...
<END>
```

Figure B.1. hyperbaton / APE: the refiner echoed APE's induction template; no answer-format line, so validation accuracy fell to zero (abridged).

```
<START>
### Task Overview You are a Senior Prompt
Engineer ... ### Prompt Under Evaluation
...
### Seven-Criteria Evaluation Rubric
...
<END>
```

Figure B.2. ruin_names / PromptWizard: the refiner echoed the evaluation meta-prompt itself; the round scored 0.0 (abridged).

APPENDIX. PIPELINE DEFECTS AND SERVING CONFIGURATION

Defects found while running the pipeline against a live model. Magnitudes are the symptoms recorded at the time. Items 1-8 were fixed before any number reported in Section 5; items 9-11 were found during this revision. Earlier pre-pilot fixes (a per-seed data-split collision, an unseeded random generator, mismatched scoring minibatches in APE, and swallowed API errors) are recorded in the repository's REBUILD.md.

TABLE 4: Table C1. Defects found in the pipeline, their observed symptom and the fix.

#	Defect	Observed symptom	Fix
1	Response truncated by an under-provisioned context	PromptWizard prompts left too little context: responses cut mid-sentence, 0/50 on a task	Explicit max new tokens (1024) an context 8192
2	Answer normalization gap	Bare letter 'A' scored wrong against gold '(A)'	Strip parentheses and case on both sides
3	Delimiter convention absent from expert exemplars	Expert few-shot scored exactly 0.0 on hyperbaton (whole response wrapped)	Append a bare tag after each exemplar's conclusion
4	Missing delimiters crashed the loop	IndexError ended every remaining condition of a task	Fallback to the unrefined prompt (now logged as a outcome)
5	Echo of the rubric meta-prompt	ruin_names / PromptWizard candidate scored 0.0	Marker-phrase echo guard
6	Echo of APE's induction template; unscored baseline	hyperbaton / APE candidate scored 0.0 and was accepted because the incumbent score started at minus infinity	Score P0 first; accept only on strict improvement
7	Evaluation template mismatch	Same prompt text scored 0.07 through one template and 0.515 through another (trailing whitespace changes greedy decoding)	Score every condition throug the base optimizer's own template
8	Search-time scoring omitted the answer format	APE/ProTeGi candidates scored near 0 during search, so selection ran on noise	Bake the answer-format line into scored instructions
9	Rubric text inconsistent	'five criteria', 'all 6 criteria', 'X/25' for a seven-criterion, 35-point rubric	Corrected text fo V1-V3; original kept as V2L
10	Refinement rounds not independent	Same input, temperature 0, seven times	Independent sampling and chained variants (Section 3.2)
11	APE search selected a degenerate prompt	hyperbaton P0 is APE's own induction template	Left in place and disclosed; same code as the pilot

APPENDIX. THE ORIGINAL EVALUATION (EXPLORATORY; VALIDATION LEAK)

The first evaluation used GPT-3.5-turbo as target and GPT-4o as refiner on all BBH tasks. It is reproduced here only because the manuscript's framing referred to it, and Reviewer comments asked for its numbers. Its validation stage scored candidates on the optimizer's own training examples and its loop never scored the unrefined prompt (Section 3.2), single runs at temperature 0. The numbers are exploratory, are not evidence for MPIR, and were not used to compute any result in Section 5. The Gemini 3.5 Flash-Lite check used the same model as target and refiner, with the same defects; its baseline already averaged 92%, leaving little headroom, and it was not analysed further.

The three pilot tasks ranked 1, 3 and 6 of 23 by the original PW+MPIR gain (hyperbaton +21.8, penguins_in_a_table +7.6, ruin_names +5.8 points). Selecting tasks on the maximum of a noisy, leaked statistic invites winner's-curse effects, which is why Tier 2 of the revised design draws tasks at random.

APPENDIX. ROUND-LEVEL RECORD OF THE PRIMARY VARIANT

Per cell (task / optimizer, seed 42) and round, the outcome of the chained variant V2 with MPIR seed 1

TABLE 5: Table C2. Serving configuration.

Item	Value
Inference engine	vLLM 0.28.0 (V1 engine, model runner V2 disabled, flashinfer sampler disabled), PyTorch 2.13.0+cu132
Target and default refiner	Qwen/Qwen3-1.7B, revision 70d244c, bf16, no quantization
Capability-experiment refiners	Qwen/Qwen3-4B-AWQ (74d4bd2) and Qwen/Qwen3-8B-AWQ (4da05a8), 4-bit AWQ
Context and limits	max model length 8192; max new tokens 1024 (target), 1500 / 3000 (meta calls); thinking disabled
Decoding	target: temperature 0; refiner: temperature 0.7 (0 for V0); seeds recorded per call
Server flags	–gpu-memory-utilization 0.80 –max-num-seqs 96 –seed 0; prefix caching on
Hardware	NVIDIA RTX 5000 Ada Generation Laptop GPU, 16 GB, driver 596.58; WSL2 (Ubuntu 24.04, kernel 5.15.167.4) on Windows 11
Non-determinism	Continuous batching; identical prompts are re-scored to measure it (Section 5.7)

(F fallback, E echo, D duplicate, V validated with its validation accuracy), then the baseline validation accuracy of P0 and the judge's rubric total out of 35 at round 1. Table E2 gives the same for the independent variant V1. Full logs for every variant, seed and round, including candidate texts and judge outputs, are in results/revision/rounds.

APPENDIX. PROTOCOL AND DEVIATIONS

The decision rules of Section 4.1 were fixed in REVISION_PROTOCOL.md before any revised experiment ran (commit 8fd1397). Dated amendments, all made before results were inspected: serving stack and measured throughput (about 1.2k tokens/s at first, then improved by fixing KV-cache thrash), which reduced the compute plan; oracle test scoring of every validated candidate only for MPIR seed 1; one validation score per candidate in G1; V2L with one seed and no oracle scoring; plain temperature-0.7 sampling without top-p; Tier-1 seed-42 cells run only in G2. The protocol and every amendment are in the repository; none is a registry entry.

ACKNOWLEDGMENT

The authors thank the reviewers for their constructive comments on earlier drafts of this manuscript. Generative AI tools were used to assist with improving the grammar and readability of the manuscript text and to generate initial code snippets during development; all AI-assisted text and code were reviewed, verified, and adapted by the authors, who take full responsibility for the manuscript's content, analyses, and conclusions.

REFERENCES

[1] D. Vilar, M. Freitag, C. Cherry, J. Luo, V. Ratnakar, and G. Foster, "Prompting PaLM for translation: Assessing strategies and performance," in Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2023, pp. 15 406–15 427.

[2] T. Goyal, J. J. Li, and G. Durrett, "News summarization and evaluation in the era of GPT-3," arXiv preprint arXiv:2209.12356, 2022.

[3] E. Kamalloo, N. Dziri, C. Clarke, and D. Rafiei, "Evaluating open-domain question answering in the era of large language models," in Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2023, pp. 5591–5606.

[4] L. Qin, Q. Chen, X. Feng, Y. Wu, Y. Zhang, Y. Li, M. Li, W. Che, and P. S. Yu, "Large language models meet NLP: A survey," Frontiers of Computer Science, 2025, arXiv:2405.12819.

[5] F. Errica, D. Sanvito, G. Siracusano, and R. Bifulco, "What did I do wrong? Quantifying LLMs' sensitivity and consistency to prompt engineering," in Proc. 2025 Conf. Nations of the Americas Chapter of the Assoc. Comput. Linguistics: Human Language Technologies (Volume 1: Long Papers), 2025, pp. 1543–1558.

[6] M. Sclar, Y. Choi, Y. Tsvetkov, and A. Suhr, "Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting," in The Twelfth International Conference on Learning Representations, 2024.

[7] P. Colombo, T. P. Pires, M. Boudiaf, D. Culver, R. Melo, C. Corro, A. F. T. Martins, F. Esposito, V. L. Raposo, S. Morgado, and M. Desa, "Saullm-7b: A pioneering large language model for law," 2024.

[8] X. Zhang, C. Tian, X. Yang, L. Chen, Z. Li, and L. R. Petzold, "AlpaCare: Instruction-tuned large language models for medical application," arXiv preprint arXiv:2310.14558, 2023.

[9] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in Proceedings of the 36th International Conference on Neural Information Processing Systems (Article 1800), 2022.

[10] A. Kong, S. Zhao, H. Chen, Q. Li, Y. Qin, R. Sun, X. Zhou, E. Wang, and X. Dong, "Better zero-shot reasoning with role-play prompting," in Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), 2024, pp. 4099–4113.

[11] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, D. Amodei et al., "Language models are few-shot learners," in Proceedings of the 34th International Conference on Neural Information Processing Systems (Article 159), 2020.

[12] J. D. Zamfirescu-Pereira, R. Y. Wong, B. Hartmann, and Q. Yang, "Why Johnny can't prompt: How non-AI experts try (and fail) to design LLM prompts," in Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems, 2023.

[13] L. Reynolds and K. McDonell, "Prompt programming for large language models: Beyond the few-shot paradigm," in Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems, 2021.

[14] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, "Large language models are human-level prompt engineers," in The Eleventh International Conference on Learning Representations, 2023.

[15] R. Pryzant, D. Iter, J. Li, Y. T. Lee, C. Zhu, and M. Zeng, "Automatic prompt optimization with "gradient descent" and beam search," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023.

[16] Q. Guo, R. Wang, J. Guo, B. Li, K. Song, X. Tan, G. Liu, J. Bian, and Y. Yang, "Connecting large language models

TABLE 6: Table D1. Original GPT-3.5-turbo evaluation (accuracy %, single run, leaked validation split) and Gemini 3.5 Flash-Lite check. PW = PromptWizard; Diff = PW+MPIR minus PW. Exploratory only.

Task	Zero-shot	Expert	PW	PW+MPIR	Diff	APE	APE+MPIR	ProTeGi	ProTeGi+MPIR	Gem. PW	Gem. MPIR
hyperbaton	74.6	84.4	62.2	84.0	+21.8	82.7	85.8	80.9	88.9	100	100
disambiguation_qa	55.1	69.3	61.7	62.2	+0.5	69.8	68.0	73.8	67.6	77	80
causal_judgement	53.7	59.9	59.3	56.8	-2.5	58.0	58.6	61.1	58.6	70	75
date_understanding	55.6	80.0	71.1	74.2	+3.1	87.6	84.4	88.0	85.8	95	96
penguins_in_a_table	45.5	74.4	75.0	82.6	+7.6	81.0	84.3	79.3	76.9	100	98
boolean_expression	81.8	91.6	92.0	94.2	+2.2	90.7	93.8	90.7	92.9	100	100
object_counting	59.1	88.0	91.5	92.4	+0.9	92.0	93.3	88.4	93.3	100	100
word_sorting	82.6	65.3	80.0	81.3	+1.3	69.3	80.0	68.4	66.7	92	92
logical_deduction	45.3	59.4	44.9	50.8	+5.9	61.9	59.7	60.7	59.3	99	96
salient_translation_error_detection	44.0	54.7	50.7	52.0	+1.3	55.1	54.7	52.9	49.3	75	74
geometric_shapes	29.3	64.4	57.8	49.3	-8.5	62.7	61.3	61.8	60.9	86	83
snarks	45.0	72.5	63.3	65.4	+2.1	61.4	68.6	75.8	73.2	90	90
temporal_sequences	33.3	17.3	44.4	38.2	-6.2	26.7	86.7	84.0	86.2	100	100
web_of_lies	47.5	80.9	52.4	52.9	+0.5	80.0	73.3	81.3	80.9	100	100
navigate	49.3	93.8	65.3	68.0	+2.7	95.6	92.9	95.6	97.8	100	99
reasoning_about_colored_objects	54.2	83.1	56.0	68.4	+12.4	82.7	82.7	82.7	83.6	100	100
sports_understanding	78.2	93.8	79.6	86.2	+6.6	95.1	96.4	93.3	94.7	91	86
multistep_arithmetic_two	33.3	84.9	51.1	51.5	+0.4	86.2	84.9	83.6	82.7	97	94
ruin_names	54.7	69.3	66.2	72.0	+5.8	58.7	68.4	58.7	77.8	88	87
movie_recommendation	66.2	79.1	73.3	66.7	-6.6	76.9	79.1	76.0	80.9	95	96
formal_fallacies	53.8	51.6	53.3	53.3	+0.0	52.4	52.9	55.1	56.9	99	99
dyck_languages	1.7	16.9	14.2	12.9	-1.3	24.4	34.2	20.9	32.4	72	75
tracking_shuffled_objects	22.4	63.8	69.8	65.5	-4.3	63.0	58.2	61.6	60.9	100	100
Average	50.7	69.5	62.4	64.4	+2.0	70.2	74.0	72.8	74.3	92	92

with evolutionary algorithms yields powerful prompt optimizers," in The Twelfth International Conference on Learning Representations, 2024.

[17] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," in The Twelfth International Conference on Learning Representations, 2024.

[18] E. Agarwal, R. Magazine, J. Singh, V. Dani, T. Ganu, and A. Nambi, "Promptwizard: Optimizing prompts via task-aware, feedback-driven self-evolution," in Findings of the Association for Computational Linguistics: ACL 2025, 2025, pp. 19 974–20 003.

[19] K. Ramnath, K. Zhou, S. Guan, S. S. Mishra, X. Qi, Z. Shen, S. Wang, S. Woo, S. Jeoung, Y. Wang, H. Wang, H. Ding, Y. Lu, Z. Xu, Y. Zhou, B. Srinivasan, Q. Yan, Y. Chen, H. Ding, P. Xu, and L. L. Cheong, "A systematic survey of automatic prompt optimization techniques," pp. 33 066–33 098, 2025.

[20] M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, P. Lu, Z. Huang, C. Guestrin, and J. Zou, "Optimizing generative ai by backpropagating language model feedback," Nature, vol. 639, pp. 609–616, 2025.

[21] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, "Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing," ACM Computing Surveys, vol. 55, 2023.

[22] Q. Dong et al., "A survey on in-context learning," in Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024, pp. 1107–1128.

[23] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer, "Rethinking the role of demonstrations: What makes in-context learning work?" in Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, 2022, pp. 11 048–11 064.

[24] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in Proceedings of the 36th International Conference on Neural Information Processing Systems (Article 1613), 2022.

[25] X. Wang and D. Zhou, "Chain-of-thought reasoning without prompting," in Proceedings of the 38th International Conference on Neural Information Processing Systems, 2024.

[26] B. Wang, S. Min, X. Deng, J. Shen, Y. Wu, L. Zettlemoyer, and H. Sun, "Towards understanding chain-of-thought prompting: An empirical study of what matters," in Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2023, pp. 2717–2739.

[27] J. Kim, N. Yang, and K. Jung, "Persona is a double-edged sword: Mitigating the negative impact of role-playing prompts in zero-shot reasoning tasks," 2024, arXiv preprint.

[28] M. Zheng, J. Pei, L. Logeswaran, M. Lee, and D. Jurgens, "When "a helpful assistant" is not really helpful: Personas in system prompts do not improve performances of large language models," in Findings of the Association for Computational Linguistics: EMNLP 2024, 2024, pp. 15 126–15 154.

[29] H. S. Zheng, S. Mishra, X. Chen, H.-T. Cheng, E. H. Chi, Q. V. Le, and D. Zhou, "Take a step back: Evoking reasoning via abstraction in large language models," in The Twelfth International Conference on Learning Representations, 2024.

[30] J. Coda-Forno, M. Binz, J. X. Wang, and E. Schulz, "Cogbench: A large language model walks into a psychology lab," in Proceedings of the 41st International Conference on Machine Learning, 2024.

[31] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," Transactions of the Association for Computational Linguistics, vol. 12, pp. 157–173, 2024.

[32] Y. Yu, H. Jiang, X. Luo, Q. Wu, C.-Y. Lin, D. Li, Y. Yang, Y. Huang, and L. Qiu, "Mitigate position bias in llms via scaling a single hidden states channel," in Findings of the Association for Computational Linguistics: ACL 2025, 2025, pp. 6092–6111.

[33] Microsoft, "Prompt engineering techniques," Microsoft Foundry documentation, Microsoft Learn, 2026, page dated May 13, 2026. Accessed Oct. 20, 2026. Archived (earlier URL): https://web.archive.org/web/20251112071324/https://learn.microsoft.us/AZURE/AI-FOUNDRY/OPENAI/CONCEPTS/PROMPT-ENGINEERING. [Online]. Available: https://learn.microsoft.com/en-us/azure/foundry/openai/concepts/prompt-engineering

[34] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in The Eleventh International Conference on Learning Representations, 2023.

[35] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "Dspy: Compiling declarative language model calls into state-of-the-art pipelines," in The Twelfth International Conference on Learning Representations, 2024.

[36] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement

with self-feedback," in Thirty-seventh Conference on Neural Information Processing Systems, 2023.

[37] N. Shinn, F. Cassano, A. Gopinath, K. R. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in Thirty-seventh Conference on Neural Information Processing Systems, 2023.

[38] W. Cui, J. Zhang, Z. Li, H. Sun, D. Lopez, K. Das, B. A. Malin, and S. Kumar, "Heuristic-based search algorithm in automatic instruction-focused prompt optimization: A survey," in Findings of the Association for Computational Linguistics: ACL 2025, 2025, pp. 22 093–22 111.

[39] Q. Ye, M. Ahmed, R. Pryzant, and F. Khani, "Prompt engineering a prompt engineer," in Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 355–385.

[40] M. Yang, M. Li, Y. Li, Z. Chen, C. Gao, J. Zhang, Y. Li, and F. Feng, "Dual-phase accelerated prompt optimization," in Findings of the Association for Computational Linguistics: EMNLP 2024, 2024, pp. 12 163–12 173.

[41] K. Mayilvaghanan, V. Nathan, and A. Kumar, "PROPEL: Prompt optimization with expert priors for small and medium-sized LLMs," in Proceedings of the 4th International Workshop on Knowledge-Augmented Methods for Natural Language Processing, 2025, pp. 272–302.

[42] A. Chakma, M. Surdeanu, and E. Blanco, "Two-stage prompt optimization for few-shot relation extraction: From reasoning-guided search to gradient-guided refinement," 2026.

[43] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab, "GEPA: Reflective prompt evolution can outperform reinforcement learning," International Conference on Learning Representations (ICLR), Oral, 2026.

[44] M. Singh, V. Yadav, and E. Blanco, "Error taxonomy-guided prompt optimization," 2026.

[45] Y. Liu, J. Xu, L. L. Zhang, Q. Chen, X. Feng, Y. Chen, Z. Guo, Y. Yang, and P. Cheng, "Beyond prompt content: Enhancing llm performance via content-format integrated prompt optimization," 2025.

[46] A. Nair, A. Banerjee, L. Mombaerts, M. Hagen, and T. Borogovac, "Tournament of prompts: Evolving LLM instructions through structured debates and Elo ratings," 2025, workshop venue (KDD 2025) UNVERIFIED; cite as arXiv preprint unless confirmed.

[47] H.-C. Lin, B. M. Ruppik, C. van Niekerk, C.-H. Shen, M. Heck, N. Lubis, R. Vukovic, S. Feng, and M. Gasic, "Prompt reinforcing for long-term planning of large language models," 2025.

[48] S. S. S. Das, R. Kamoi, B. Pang, Y. Zhang, C. Xiong, and R. Zhang, "GReaTer: Gradients over reasoning makes smaller language models strong prompt optimizers," in The Thirteenth International Conference on Learning Representations, 2025.

[49] J. Shim, D. W. Kim, S. W. Kim et al., "POaaS: Minimal-edit prompt optimization as a service to lift accuracy and cut hallucinations on on-device sLLMs," 2026, accepted at FEVER 2026 (per arXiv comment).

[50] Á. Pándy, R. Lakatos, and A. Hajdu, "Error-driven prompt optimization for arithmetic reasoning," IEEE Access, vol. 14, pp. 62 570–62 583, 2026.

[51] S. Badhe and D. Shah, "Prompt-level distillation: A non-parametric alternative to model fine-tuning for efficient reasoning," 2026, accepted at ACL 2026 Industry Track (per arXiv comment); proceedings pages UNVERIFIED.

[52] D. Shah, S. Badhe, N. Kathrotia, and P. Tiwari, "CROP: Token-efficient reasoning in large language models via regularized prompt optimization," 2026, accepted at ICLR 2026 Workshop on Logical Reasoning of Large Language Models (per arXiv comment).

[53] Z. Zhu, H. Zhou, Z. Feng, T. Li, C. J. J. Deryl, M. L. Onn, G. W. Ng, and K. Mao, "Rethinking prompt optimizers: From prompt merits to optimization," 2025, no peer-reviewed venue found.

[54] S. M. Bsharat, A. Myrzakhan, and Z. Shen, "Principled instructions are all you need for questioning LLaMA-1/2, GPT-3.5/4," 2023, no peer-reviewed venue found.

[55] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in The Twelfth International Conference on Learning Representations, 2024.

[56] Y. Zhang, M. Khalifa, L. Logeswaran, J. Kim, M. Lee, H. Lee, and L. Wang, "Small language models need strong verifiers to self-correct reasoning," in Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 15 637–15 653.

[57] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in Advances in Neural Information Processing Systems 36 (Datasets and Benchmarks Track), 2023.

[58] P. Wang, L. Li, L. Chen, Z. Cai, D. Zhu, B. Lin, Y. Cao, L. Kong, Q. Liu, T. Liu, and Z. Sui, "Large language models are not fair evaluators," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2024, pp. 9440–9450.

[59] A. Panickssery, S. R. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," 2024, arXiv preprint; no peer-reviewed venue found (NeurIPS 2024 listing UNVERIFIED).

[60] X. Wang, B. Ma, C. Hu, L. Weber-Genzel, P. Röttger, F. Kreuter, D. Hovy, and B. Plank, ""my answer is C": First-token probabilities do not match text answers in instruction-tuned language models," in Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 7407–7416.

[61] C. Zheng, H. Zhou, F. Meng, J. Zhou, and M. Huang, "Large language models are not robust multiple choice selectors," in The Twelfth International Conference on Learning Representations, 2024.

[62] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," Journal of Machine Learning Research, vol. 22, no. 164, pp. 1–20, 2021.

[63] J. Dodge, S. Gururangan, D. Card, R. Schwartz, and N. A. Smith, "Show your work: Improved reporting of experimental results," in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, 2019, pp. 2185–2194.

[64] X. Bouthillier, P. Delaunay, M. Bronzi, A. Trofimov, B. Nichyporuk, J. Szeto, N. Mohammadi Sepahvand, E. Raff, K. Madan, V. Voleti, S. Ebrahimi Kahou, V. Michalski, T. Arbel, C. Pal, G. Varoquaux, and P. Vincent, "Accounting for variance in machine learning benchmarks," in Proceedings of Machine Learning and Systems 3 (MLSys 2021), 2021.

[65] R. Dror, G. Baumer, S. Shlomov, and R. Reichart, "The hitchhiker's guide to testing statistical significance in natural language processing," in Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2018, pp. 1383–1392.

[66] D. Card, P. Henderson, U. Khandelwal, R. Jia, K. Mahowald, and D. Jurafsky, "With little power comes great responsibility," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, 2020, pp. 9263–9274.

[67] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze, A. Liu, C. Si, Y. Li, A. Gupta, H. Han, S. Schulhoff, P. S. Dulepet, S. Vidyadhara, D. Ki, S. Agrawal, C. Pham, G. C. Kroiz, F. Li, H. Tao, P. Resnik et al., "The prompt report: A systematic survey of prompting techniques," 2024.

[68] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A systematic survey of prompt engineering in large language models: Techniques and applications," 2024.

[69] L. Boonstra, "Prompt engineering," Google whitepaper, Kaggle, 2025, archived:

https://web.archive.org/web/20260905081837/https://www.kaggle.com/whitepaper-prompt-engineering. Publication month UNVERIFIED. [Online]. Available: https://www.kaggle.com/whitepaper-prompt-engineering
[70] Anthropic, "Prompt engineering overview," Claude Developer Platform documentation, 2025, accessed Oct. 20, 2026. Page revised since the version used for the rubric; archived: https://web.archive.org/web/20261019210255/https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview. [Online]. Available: https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview
[71] OpenAI, "Best practices for prompt engineering with the OpenAI API," OpenAI Help Center, 2025, archived: https://web.archive.org/web/20260915104022/https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api. Live page returned HTTP 403 on Oct. 20, 2026; no access date asserted. [Online]. Available: https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api
[72] A. Yang et al., "Qwen3 technical report," 2025.

LINH NGUYEN Linh Nguyen is a Master's student in the Master of Artificial Intelligence program at RMIT University Vietnam, Ho Chi Minh City, Vietnam.

QUANG-VINH DANG Quang-Vinh Dang is a Senior Lecturer at British University Vietnam, Hung Yen, Vietnam.

MINH NGOC DINH Minh Ngoc Dinh is Dean of the School of Information Technology and Data Science, Millenia Education, Ho Chi Minh City, Vietnam.

THUY NGUYEN Thuy Nguyen is a Senior Lecturer at RMIT University Vietnam, Ho Chi Minh City, Vietnam.